RHOSO 18.0 on OpenShift 4.20 — Deployment and Troubleshooting Report

Compiled from the deployment session

2026-10-01

RHOSO 18.0 on OpenShift 4.20 — Deployment and Troubleshooting Report
1. Objective and environment
2. Control-plane bring-up
3. Compute node (data plane) — initial network and dataplane objects
4. Deep troubleshooting of the data-plane deployment
5. Final state
6. Known caveats carried forward

RHOSO 18.0 on OpenShift 4.20 — Deployment and Troubleshooting Report

1. Objective and environment

The goal was to deploy **Red Hat OpenStack Services on OpenShift (RHOSO) 18.0** control plane on a 3-node compact OpenShift cluster, and then add one pre-provisioned bare-metal **RHEL 9.4** host as an OpenStack compute node (data plane).

Component	Version / Value
OpenShift	

Component	Version / Value
	4.20.15, 3 nodes acting as control-plane + master + worker
Masters	`openshift-master-1/2/3` (100.96.39.21–23)
openstack-operator	1.22.0 (RHOSO 18.0.22), channel `stable-v1.0`
Compute host	`100.96.39.20` (`openshift-master-0`), RHEL 9.4, single NIC `eno2` used for OpenStack networks
Repository	`https://github.com/GuillermoMM/rhoso18-openshift-deploy` (public)

All manifests, the `deploy.sh` runner, and this report live in that repository. Real secrets are excluded from git; only `*.example.yaml` templates are committed.

---

2. Control-plane bring-up

2.1 OLM / operator installation

- **Problem:** the OpenStack operator subscription failed with `ResolutionFailed` — no `openstack-operator` package in the catalog.
- **Reason:** the cluster was originally OpenShift 4.22.11, which RHOSO 18.0 does not support. RHOSO 18.0.21+ supports only OCP 4.18 and 4.20.
- **Fix:** rebuilt the cluster on OpenShift **4.20.15**. The `openstack-operator` subscription (channel `stable-v1.0`, source `redhat-operators`) then resolved and installed (CSV `openstack-operator.v1.22.0`, Succeeded).

2.2 cert-manager

- **Problem:** the `OpenStackControlPlane/OpenStack` instance failed with `no matches for kind "Issuer" cert-manager.io/v1`.
- **Reason:** cert-manager was not installed. A first attempt used the wrong package name (`cert-manager-operator`, channel `stable`) and no `OperatorGroup`.
- **Fix:** installed the correct package `openshift-cert-manager-operator`, channel `stable-v1`, with an `OperatorGroup` (`targetNamespaces: []`) in namespace `cert-manager-operator`.

CSV `cert-manager-operator.v1.20.x` succeeded, and the cert-manager CRDs (`issuers`, `certificates`) appeared.

2.3 Control-plane / etcd outage

- **Problem:** after fixing cert-manager, the webhook still failed with `ValidatingAdmissionWebhook failed to complete validation in 13s`.
- **Reason:** the OpenShift control plane itself was unhealthy: `kube-apiserver` and `kube-controller-manager` had 0 static pods, and `etcd` on master-2 was crash-looping, left over from a failed 4.20.15 upgrade.
- **Fix:** the user recovered cluster health independently. All `ClusterOperators` were re-verified `Available=True, Degraded=False` before continuing.

2.4 FIPS and iSCSI CHAP

- FIPS was confirmed disabled on all nodes (`/proc/sys/crypto/fips_enabled=0`). FIPS can only be enabled at OpenShift **install time**; a post-install `MachineConfig` toggle is not supported and was explicitly avoided.
- An iSCSI CHAP hardening `MachineConfig` (`99-master-iscsi-chap-algs`) was applied, rewriting `/etc/iscsi/iscsid.conf` so `node.session.auth.chap_algs = SHA3-256,SHA256`. This rolled across all three masters with a reboot each (serialized by the MCP), briefly making master-2 `NotReady` before it recovered.

2.5 NMState and node networking (NNCP)

- **Problem:** `oc get nns` failed with `the server doesn't have a resource type "nns"`.
- **Reason:** the NMState operator was not installed.
- **Fix:** installed `kubernetes-nmstate-operator` (channel `stable`, namespace `openshift-nmstate`, OwnNamespace-only `OperatorGroup`) and created the `NMState` singleton CR.
- **Problem:** the `NodeNetworkConfigurationPolicy` (NNCP) failed with `Controller interface octbr is holding unknown port enp2s0f1.24`.
- **Reasons (two separate bugs):**
  1. The Octavia VLAN interface was named `octavia`, but the bridge's `port` list referenced `enp2s0f1.24` instead of that logical name.
  2. `enp2s0f1` does not exist on the real hardware — the actual NICs are `eno1`–`eno6np1`/`ens3f0np0`/`ens3f1np1`. `eno1` carries the cluster's own `br-ex` network and was explicitly avoided. `eno2` was confirmed (via `ip link`, carrier up on all three masters) as the correct uplink for the OpenStack networks.

- **Fix:** rewrote the NNCPs to use `eno2` as the base interface for all VLANs (internalapi 20, storage 21, tenant 22, octavia 24, designate 25, designateext 26), with `eno2` itself carrying the untagged ctlplane address (192.168.122.10/.11/.12 for master-1/2/3). The bridge port was corrected to reference the `octavia` interface name.
- **Secondary bug found and fixed later:** master-2 and master-3's NNCPs originally **duplicated master-1's IP** (`.10`) on every network. Fixed by giving master-2 `.11` and master-3 `.12` on all networks.

2.6 MetalLB

- **Problem:** `IPAddressPool` application failed with `no matches for kind "IPAddressPool" in version "metallb.io/v1beta1"`.
- **Reason:** the MetalLB operator was not installed at all (no CRDs, no `metallb-system` namespace).
- **Fix:** installed `metallb-operator` (channel `stable`, AllNamespaces), created the `MetalLB` instance, then applied the `IPAddressPools` for `internalapi`, `ctlplane`, `storage`, `tenant`.
- **Problem:** the `ctlplane` `L2Advertisement` referenced interface `enp6s0` (a placeholder copied from Red Hat documentation examples), which does not exist on the masters. This silently prevented MetalLB from ever announcing the `ctlplane` VIPs (DNS, etc.) over ARP.
- **Fix (initially only in the repo, see §4.3 for when it was actually applied to the live cluster):** changed the `ctlplane` `L2Advertisement` interface to `eno2`.

2.7 OpenStackControlPlane: storage and telemetry

- **Problem:** `Telemetry: ... customresourcedefinitions ... "monitoringstacks.monitoring.rhobs" not found`.
- **Reason:** `spec.telemetry.enabled: true` requires the **Cluster Observability Operator**, which was not installed.
- **Fix:** installed `cluster-observability-operator` (channel `stable`, AllNamespaces), which provides the `monitoringstacks.monitoring.rhobs` CRD.
- **Problem:** Prometheus (`prometheus-metric-storage-0`) could not be created: `storage class "your-RHOCP-storage-class" does not exist`.
- **Reason:** `spec.storageClass` was left as the literal placeholder from the Red Hat docs; the cluster had no `StorageClass` at all.
- **Fix:** installed **LVM Storage** (`lvms-operator`, channel `stable-4.20`, OwnNamespace), created an `LVMCluster` using a free 447 GB SSD on each node (`/dev/disk/by-path/pci-0000:5c:00.0-scsi-0:1:0:1` — a stable path, since `/dev/`

`sda`/`sdb` letters differ between nodes), which produced `StorageClass lvms-vg1`. Updated `spec.storageClass: lvms-vg1` in the `OpenStackControlPlane`.

- **Problem:** the Prometheus pod stayed `Init:0/1`, event: `error adding container to network "ctlplane": Link not found`.
- **Reason:** the `ctlplane` `NetworkAttachmentDefinition` (Multus) still used the placeholder `"master": "enp6s0"`.
- **Fix:** changed it to `eno2`, then deleted the stuck pods so they were recreated.
- **Cosmetic bug found and fixed:** the RabbitMQ `loadBalancerIPs` annotations in `openstack_control_plane.yaml` had stray Unicode circled digits (`172.17.0.85` ⑤, `172.17.0.86` ⑥) pasted in from a documentation callout. These were removed in the repo; **the live cluster object still had them until §4.3**, which silently prevented MetalLB from ever assigning external IPs to the `rabbitmq`/`rabbitmq-cell1` services.

2.8 Repository creation

All of the above manifests, plus a `deploy.sh` script that applies each step and runs its verification command, were committed to a new **public** GitHub repository, `GuillermoMM/rhoso18-openshift-deploy`. Real secrets (`osp-secret`, SSH keys, registry/subscription credentials) were excluded via `.gitignore`; only placeholder `*.example.yaml` templates are tracked. The repository content was scanned for secrets/PII before the initial push.

---

3. Compute node (data plane) — initial network and dataplane objects

3.1 Can OpenShift nodes be computes?

No — RHOSO's control plane runs only as pods on OpenShift. Compute (data-plane) nodes must be separate RHEL 9.4 hosts managed by Ansible through `OpenStackDataPlaneNodeSet` / `OpenStackDataPlaneDeployment`.

3.2 Network interface configuration on the bare-metal host

The new host, `100.96.39.20`, needed the same four networks as the masters on its single NIC `eno2`:

Network	VLAN	IP assigned
ctlplane	untagged	192.168.122.100/24
internalapi	20	172.17.0.100/24
storage	21	172.18.0.100/24
tenant	22	172.19.0.100/24

These were first configured manually with `nmcli` (one VLAN sub-interface per network) to confirm connectivity, each time checking the address wasn't already in use (`arping`) before assigning it.

Important lesson learned: before creating the `OpenStackDataPlaneNodeSet`, the manually-created VLAN interfaces were removed and `eno2` was reverted to its original (DHCP-pending, no address) state, on the assumption that `os-net-config`/nmstate would configure everything from scratch. This was wrong: `OpenStackDataPlaneNodeSet` uses `preProvisioned: true`, which means **Ansible must already be able to SSH to the ctlplane IP before it does anything** — the "redhat" play connects first, and only a later `configure-network` step reconfigures the NIC. Removing the IP caused `"No route to host"` SSH failures that retried forever. The fix was to re-apply the static ctlplane IP (`192.168.122.100/24` on `eno2`) and leave it in place until the Ansible run itself took over the interface.

3.3 NetConfig was incomplete

- **Problem:** the `OpenStackDataPlaneNodeSet` failed IP reservation — `NetConfig` only defined the `ctlplane` network.
- **Fix:** added `internalapi` (VLAN 20, 172.17.0.0/24), `storage` (VLAN 21, 172.18.0.0/24) and `tenant` (VLAN 22, 172.19.0.0/24) networks to `NetConfig`, each with an allocation range (`.100–.250`) that didn't overlap the masters' static IPs, the NAD pod range, or the MetalLB pool.
- **Schema note:** the `vlan` field belongs under `spec.networks[].subnets[]`, **not** at the network level — an initial attempt placed it incorrectly and was corrected before applying.

3.4 NodeSet rewritten for the real hardware

The pasted example `OpenStackDataPlaneNodeSet` assumed a bonded two-NIC host with `cloud-admin` SSH access and PVC-backed logs. It was rewritten for this single-NIC, single-user host:

Change	Reason
Removed `edpm-compute-1`	Only one host exists
`ansibleUser: root`	`cloud-admin` does not exist on this host
`defaultRoute: false` on ctlplane	Avoids adding a second default route that would break SSH via `eno1`
`nic1` mapped to `eno2`'s real MAC, no bond	Single physical NIC
Removed `extraMounts`/PVC reference	No such PVC existed
Filled real `rhc_repositories`	Placeholders would not resolve
Real NTP server (`2.rhel.pool.ntp.org`)	Matches what the OpenShift nodes already use

The Ansible SSH public key (`dataplane-ansible-ssh-private-key-secret`) was appended to `root`'s `authorized_keys` on the host, and login was verified before creating the `OpenStackDataPlaneDeployment`.

---

4. Deep troubleshooting of the data-plane deployment

This was the longest part of the session. Each failure below was diagnosed down to the exact Ansible task and host-level root cause before being fixed — in several cases by pulling the exact role source out of the Ansible Execution Environment (AEE) container image used by the operator.

4.1 Missing RHOSO content repository (subscription gap)

- **Symptom:** the `redhat/download-cache/bootstrap` Ansible plays repeatedly failed trying to install `crudini`, `rhoso-release`, `openstack-network-scripts`, then `os-net-config`.
- **Root cause:** the host's Red Hat subscription only entitles `rhoso-tools-18-for-rhel-9-x86_64-rpms` (client tooling), **not** the real RHOSO 18.0 content repository that ships `rhoso-release` and `os-net-config`. Confirmed by checking the CDN URLs (`.../rhoso-tools/18/os`, never `.../rhoso/18/os`) and that `subscription-manager repos --list` never showed the real repo under any plausible name.

- **This is an account/subscription-level gap**, not fixable from the host. It was flagged to the user as needing a corrected RHOSO 18.0 entitlement.
- **Workarounds applied to keep progressing (all clearly non-default):**
  - `crudini`: only used by the adoption-from-RHOSP-17 role, not needed for a fresh install → removed from `edpm_bootstrap_packages_bootstrap` (kept `driverctl`, `lvm2`).
  - `rhoso-release`: only adds the RHOSO yum repos, which were already configured manually → `edpm_bootstrap_release_version_package: []`.
  - `openstack-network-scripts`: only needed for the legacy "network" sysvinit service, irrelevant since NetworkManager is used → `edpm_bootstrap_legacy_network_packages: []`.
  - `openvswitch` (wrapper package): not entitled; the versioned `openvswitch3.3` package from the already-enabled `fast-datapath` repo was used instead (`edpm_ovs_packages: [openvswitch3.3]`), confirmed to provide the same `openvswitch.service` unit.
  - `os-net-config`: this one is **not optional** under the default network-configuration tool and is also missing from the entitled repos. See §4.2.

4.2 Switching to the (unsupported) nmstate network-config tool

- **Root cause:** `os-net-config` is the default `edpm_network_config_tool`, and it is unavailable (§4.1). The boolean `edpm_network_config_nmstate: true` only changes `os-net-config`'s internal provider — it does not remove the need for the `os-net-config` package itself.
- **Decision (explicitly confirmed with the user, since Red Hat marks it "experimental, unsupported"):** switched to `edpm_network_config_tool: nmstate` with `edpm_network_config_tool_nmstate_override: true`, which uses `linux-system-roles.network` directly — needing only `NetworkManager-ovs`, `nmstate`, and `openvswitch3.3`, all available in already-enabled base repos.
- **Template rewritten** from the os-net-config YAML schema (`network_config: - type: ovs_bridge ...`) to a native nmstate state (`interfaces: - type: ovs-bridge ...`), building `br-ex` as an OVS bridge with `eno2` as a trunk port and internal tagged ports for `internalapi`/`storage`/`tenant`.
- **Follow-up bug:** the template initially used the symbolic name `nic1` (a convention specific to `os-net-config`'s MAC-mapping feature). The pure-nmstate path does not translate it, causing `NmstateValueError: Ethernet interface nic1 does not exists`. Fixed by using the real interface name `eno2` directly in the template.

- **Result, verified on the host:** `br-ex` OVS bridge created with `eno2` as a system port and `internalapi`/`storage`/`tenant` as tagged internal ports (VLAN 20/21/22) — exactly the intended layout. Reboots during the deployment (`reboot-os`) were confirmed not to disturb this configuration, and management access via the separate `eno1` NIC was continuously verified throughout to avoid losing remote access.

4.3 Stale/never-applied earlier fixes to the live cluster

While investigating later connectivity problems, two fixes that had been made in the Git repository weeks earlier were discovered to have **never actually been applied to the live OpenShift cluster**: - The `ctlplane` `L2Advertisement` was still `enp6s0`, not `eno2` — this blocked MetalLB from ever announcing the ctlplane DNS VIP (`192.168.122.80`). - The RabbitMQ `loadBalancerIPs` annotations on the live `OpenStackControlPlane` still contained the stray □/□ characters — this kept `rabbitmq`/`rabbitmq-cell1`'s `LoadBalancer` services permanently `<pending>`, which later blocked `nova-compute` from ever reaching RabbitMQ.

Both were applied directly to the live cluster once discovered, each immediately resolving the symptom it was blocking (DNS VIP reachability, then RabbitMQ VIP assignment).

4.4 Freshly-imaged host needed OS updates

- **Observation:** the bare-metal host had **206 pending package updates** (kernel, `NetworkManager`, `systemd`, `podman`, `openssh`, etc.) since it was freshly imaged from ISO and never patched.
- **Risk considered:** `podman` is the container runtime for every EDPM service (nova, OVN, libvirt, etc.), so an outdated version was a real risk to the rest of the deployment.
- **Action (confirmed with the user given the host had no console/IPMI access, only network):** the running `OpenStackDataPlaneDeployment` was paused first (to avoid a `dnf` lock conflict with the live Ansible job), then `dnf -y update` was run (installing a new kernel, `5.14.0-427.127.1.el9_4`), followed by a reboot. Network state (ctlplane IP, SSH key, `eno1` management access) was verified intact afterward, and the deployment was resumed.

4.5 OVN gateway mode blocking all RHOSO LoadBalancer VIPs

- **Symptom:** none of the RHOSO APIs (keystone, nova, neutron, placement, DNS) were reachable from the compute node over TCP/UDP, even though MetalLB correctly answered ARP for their VIPs.
- **Root cause:** OpenShift's OVN-Kubernetes was running with `routingViaHost: false`. In that mode, OVN-Kubernetes does not route `LoadBalancer` traffic that arrives via secondary/Multus interfaces (exactly how the compute node reaches these VIPs, over `br-ex`/VLANs rather than the primary cluster network).
- **Fix (cluster-wide change, confirmed with the user beforehand since it affects all 3 OpenShift nodes and triggers an `ovnkube-node` rollout):**

  ```
  oc patch network.operator cluster --type=merge -p \
    '{"spec":{"defaultNetwork":{"ovnKubernetesConfig":
  {"gatewayConfig":
    {"routingViaHost":true,"ipForwarding":"Global"}}}}}'
  ```

  The `network` ClusterOperator rolled through all three nodes and returned to `Available=True, Progressing=False, Degraded=False`. Immediately afterward, both the ctlplane DNS VIP and the internalapi keystone VIP became reachable from the compute node.

4.6 DNS configuration on the compute host

Getting DNS right took several iterations: - The compute initially used only the lab's external DNS servers, which cannot resolve `*.openstack.svc` names (needed for OVN database hostnames, RabbitMQ, etc.). - Simply adding the RHOSO dnsmasq VIP (`192.168.122.80`) as a 4th nameserver silently did nothing, because **glibc's resolver only reads the first three `nameserver` lines** in `/etc/resolv.conf`. - Using a very negative `ipv4.dns-priority` to force it first also broke things, because NetworkManager treats any negative priority as **exclusive**, dropping the lab's real DNS/search domains entirely (external names like `google.com`/`cdn.redhat.com` stopped resolving). - **Final, correct configuration:** an explicit, ordered 3-entry nameserver list (`192.168.122.80`, then the lab's two real servers) set on the `br-ex-if` connection profile (the actual NetworkManager connection holding the ctlplane IP, found via `nmcli -t -f NAME,DEVICE  con show`), with `ignore-auto-dns` set on `eno1` so it stopped contributing conflicting entries, and the search-domain list (`openstack.svc os.dc-fabric-lab fss-oc.dc-fabric-lab`) covering both internal

and external lookups. - **Regression caught and fixed:** this DNS change initially dropped the lab's original search domains, which broke `hostname -f` (it depends on DNS search-domain expansion to turn the bare hostname into an FQDN). The data-plane `validate-network` Ansible step failed with `openshift-master-0 does not contain . and does not appear to be an FQDN` until the search domains were restored.

4.7 OVN SB/NB database connection bug (multi-A-record DNS)

- **Symptom:** `ovn-controller` and `ovsdb-client` both failed to connect to `ssl:ovsdbserver-sb.openstack.svc:6642` with `Address family not supported by protocol`, even though DNS resolution itself worked correctly and plain TCP to every individual resolved IP succeeded.
- **Root cause, isolated step by step:**
  - `ovsdbserver-sb`/`ovsdbserver-nb` are intentionally **headless** Kubernetes services (`ClusterIP: None`) backing a 3-node OVSDB RAFT cluster — DNS returns all 3 pod IPs as separate A records by design.
  - Direct IP connection worked perfectly via `ovsdb-client`; only the FQDN (multi-A-record) form triggered the error — isolating the bug to OVS's own DNS/stream-resolution handling of multiple A records, not to networking, certificates, or DNS correctness.
- **Workaround (flagged as lab-only, reduces HA until revisited):** pinned `ovsdbserver-sb.openstack.svc` and `ovsdbserver-nb.openstack.svc` to one healthy replica IP each in the host's `/etc/hosts`, and set `base_hosts_file = "/etc/hosts"` in `/etc/containers/containers.conf.d/99-base-hosts.conf` so every podman container (recreated fresh for each EDPM service) inherits this single-IP mapping instead of the host's own minimal default.

4.8 SELinux: a three-stage saga

1. **First symptom:** `ovn_controller`'s systemd unit failed with `Failed at step EXEC spawning ...: Permission denied` (exit 203/EXEC). The shared script `/var/local/libexec/edpm-start-podman-container` had SELinux context `container_file_t` instead of the policy-expected `var_t`/an executable type. A one-off `chcon -t bin_t` fixed it immediately, confirming the diagnosis.
2. **Regression:** the context reverted to `container_file_t` on the next service's container creation (`ovn_metadata_agent`), and kept recurring — a persistent `semanage fcontext` rule was added, which held for one cycle but still wasn't reliable enough given 2–3 more services (`libvirt`, `nova`, `telemetry`) remained to deploy.

3. **Decision (confirmed with the user, flagged as a real security posture reduction):** moved to disabling SELinux enforcement entirely on this lab compute host rather than chasing per-service label resets.
   - First attempt used `setenforce 0` + `SELINUX=permissive` in `/etc/selinux/config`. This was silently reverted back to `Enforcing` by the time the next `reboot-os` step ran (exact mechanism not conclusively identified, though Insights/`rhc` remediation tasks were investigated and ruled out as the direct cause).
   - A `grubby --update-kernel ALL --args selinux=0` kernel-level disable was then tried (confirmed with the user as one step further than "permissive"). This technically disabled SELinux (`getenforce` → `Disabled`) but **broke podman container start** with a new, different error: `failed to mount shm tmpfs ...: invalid argument`. Root cause: RHEL 9's own `/etc/selinux/config` documentation warns that the kernel `selinux=0` parameter removes the LSM hook entirely, which some tools (including podman/crun) don't handle gracefully, unlike the recommended `SELINUX=disabled` config-file setting (which keeps the LSM present with no policy loaded).
   - **Final, correct fix:** removed `selinux=0` from the kernel command line (`grubby --remove-args selinux`), set `SELINUX=disabled` in `/etc/selinux/config` (the RHEL9-recommended method), and rebooted once more. This resolved both the original exec-permission errors and the shm-mount regression.

4.9 Stale Ansible job retries vs. real failures

Several times during debugging, a `systemctl restart/start` on a broken service reported the same old `203/EXEC` failure even after the root cause had already been fixed. This turned out to be systemd's own start-rate-limiting (`StartLimitBurst`) replaying a cached failure rather than making a fresh attempt. `systemctl reset-failed <service>` before each retry was required to get a true, fresh result — this was confirmed by using `systemd-run` (a brand-new unit, unaffected by the rate limit) to prove the underlying fix had actually worked.

4.10 Final successful run

With all of the above fixed, a clean `OpenStackDataPlaneDeployment` run completed every service in order without manual intervention: `redhat → download-cache → bootstrap → configure-network → validate-network → install-os → configure-os → ssh-known-hosts → run-os → reboot-os → install-certs → ovn → neutron-metadata → libvirt → nova →`

`telemetry`, ending with `Ready=True, "Setup complete"` on both the `OpenStackDataPlaneDeployment` and the `OpenStackDataPlaneNodeSet`.

4.11 Final registration steps

- `nova-compute` came up but could not reach RabbitMQ (`socket.gaierror: No address found`) — this was the RabbitMQ VIP issue from §4.3; once the live `OpenStackControlPlane` annotation was fixed and MetalLB assigned real VIPs to `rabbitmq`/`rabbitmq-cell1`, restarting `edpm_nova_compute` connected immediately.
- The compute then appeared as an **enabled, up** `nova-compute` service, but `openstack hypervisor list` stayed empty. Root cause: Nova cells v2 requires new compute hosts to be explicitly discovered and mapped into a cell. Running `nova-manage cell_v2 discover_hosts --verbose` (inside the `nova-cell0-conductor-0` pod) created the host mapping, and the hypervisor immediately appeared: `openshift-master-0.os.dc-fabric-lab`, type `QEMU`, IP `192.168.122.100`, state `up`.
- OVN agents (`OVN Controller agent`, `OVN Metadata agent`) were confirmed `UP` for `edpm-compute-0.ctlplane.example.com` throughout.

---

5. Final state

Check	Result
`openstack compute service list`	`nova-compute` on `edpm-compute-0.ctlplane.example.com`, enabled, up
`openstack hypervisor list`	`openshift-master-0.os.dc-fabric-lab`, QEMU, 192.168.122.100, up
`openstack network agent list`	OVN Controller agent + OVN Metadata agent, both up
Resource provider (Placement)	80 vCPUs, 257 GB RAM, 69 GB disk reported

6. Known caveats carried forward

1. **Network config tool:** the compute node uses the Red-Hat-labelled experimental, unsupported `nmstate` path instead of the default `os-net-config`, because the latter's package isn't entitled.

2. **SELinux is disabled** on the compute host (not permissive, not enforcing with a working policy).
3. **OVN DB connectivity** depends on pinned single-IP `/etc/hosts` entries rather than genuine multi-replica client failover; if that specific OVSDB pod is rescheduled with a new IP, the entries must be updated manually.
4. **Subscription gap:** the host still lacks the real RHOSO 18.0 content repository (only the “tools” channel is entitled). Several package substitutions were made to work around this; a corrected subscription would allow reverting to the fully supported defaults (`os-net-config`, `rhoso-release`, the standard `openvswitch` wrapper package).
5. The RHOSO sample manifests pulled from Red Hat documentation contained several placeholder/copy-paste defects (`enp6s0`, `your-RHOCP-storage-class`, stray Unicode callout digits in IP annotations, duplicated static IPs across masters) that all had to be identified and corrected against the real hardware and cluster.